

Bottleneck Experiment Template (DP Section 5)

Day 5 · Activity 1 handout. Convert each top queue from your VSM into a specific experiment: hypothesis, test, and a *measurable* success criterion. "We'll see if it feels better" is not an experiment — it's a vibe.

The theory of constraints (why one at a time)

A system's throughput is constrained by exactly **one** bottleneck at a time. If you optimize anything other than the current bottleneck, you don't speed up the system — you just add inventory in front of the constraint. Find the current bottleneck, remove it, *then* find the next one.

The experiment structure

```
### Experiment N: <short name>
**Bottleneck addressed:** <queue from VSM>

**Hypothesis:** We believe that <doing X> will <result>
because <reasoning>.

**Test:** <what we'll do, for how long, on what scope>

**Success criterion:** <measurable outcome that decides
adopt/reject>
- Baseline: <current measurement>
- Target: <improved measurement>
- Window: <duration>

**What could go wrong:**
<2–3 risks of running this experiment>

**Who runs it:** <named person>
**By when:** <date>

**Decision after:**
- If success criterion is met: <adopt / scale / institutionalize>
- If not met: <abandon / iterate / escalate>
```

Worked example — FieldPulse

```
### Experiment 1: Auto-assign code reviewers

**Bottleneck:** Build → Code review queue (1.5 days average wait).

**Hypothesis:** We believe that auto-assigning code reviewers
based on file paths (using CODEOWNERS) will reduce time-to-first-
review from 1.5 days to  $\leq 4$  hours, because the current 1.5-day
wait is mostly "who's going to review this" decision time.

**Test:** Configure CODEOWNERS for the reconcile module; run
for 4 sprints (8 weeks); compare time-to-first-review and time-
to-merge against the 8 sprints prior.

**Success criterion:**
- Baseline: time-to-first-review p50 = 1.5 days, p90 = 3 days
- Target: p50  $\leq 4$  hours, p90  $\leq 1$  day, over the 4 sprints
- Window: 8 weeks; min sample 30 PRs

**What could go wrong:**
- Reviewers feel burdened by constant assignment → measure complaints /
opt-outs
- Auto-assigned reviewer is on PTO → fall-back to next-in-list
- One reviewer becomes overloaded → measure per-reviewer load

**Who runs it:** <eng lead> with <TPM> support
**By when:** Pilot starts next sprint (2026-05-13)

**Decision after:**
- Met: institutionalize CODEOWNERS for the whole codebase
- Not met: examine why; possibly try paired-review during dev
```

Sanity check (per experiment)

- Commit to a **specific change** — not "improve code review," but "auto-assign reviewers based on path."
- Commit to a **duration** — typically 2–4 sprints. Under 2 sprints rarely yields enough data.
- The success criterion has a **baseline, a target, and a window**.

- Would your team actually try this in the next 60 days? If not, it's too ambitious or too vague.